

Math Modeling Final - an iPhone under Tariff

Chengyi (Mark) Shen

Tufts University

Abstract. On April 3 2025, the United States imposed a new set of tariffs on many countries and caused much discussion on how the global supply chain of big companies can affect consumer prices in the United States. Apples, one of the biggest tech company that have their products manufactured around the world with many parts outsourced and produced in countries like China and Vietnam for a lower production cost, could suffer a big hit in its profit margin under this condition. This project develops a network flow model that simulates the iPhone's global supply chain, enabling analysis of how varying tariff rates affect Apple's profit margins. The model will identify cost-effective supply chain configurations under this tariff scenarios, providing insights into strategic adjustments Apple might consider with the pretext of the exact amount of tariffs imposed on April 3.

1 Introduction

iPhone, the most profitable phone product on the planet, provides a big proportion of revenue for its company. To make such intricate product profitable, the company uses many outsourcing and production chain from various countries to minimize the production cost thus maximize the profit on each iPhone. This new set of tariff is imposed on many main supplier nation of Apples and certainly will increase the cost for making an iPhone. To figure out the main question: How does the tariff affect the profit margin of an iPhone, we need to know the cost of each component and country they are coming from. The cost of an iPhone is a company secret, and the exact cost of all its parts is also not available to normal buyers. But we will use information from major new outlets like Wall Street journal, to be specific, this:

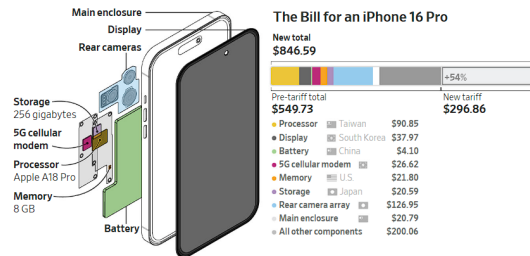


Fig. 1. Cost breakdown of an iPhone 16 pro according to WSJ

At last, we are not assuming that tariff will be charged on components, but rather on the place of manufacture as a whole; and we will assume that the data given will be the cheapest price given on the market. We will also assume that the provided cost of each component already incorporated pre-April tariff and in-between provider countries are calculated without any additional cost.

2 Methodology

We apply a minimum-cost network flow model, implemented in Python using `scipy.optimize.linprog`, to simulate the supply and delivery chain. Each component of the iPhone is modeled as a subnetwork, and all are later integrated into a final manufacturing and distribution network. There will be examinations of pricing and consumer cost on the way. The network flow model and functions will be based on class notes and from the Midterm 1 report.

3 Basic Model

Each major component is modeled as an independent supply chain network with the following structure:

- Source Node: Represents the global start of the component flow.
- Supplier Countries: Countries manufacturing the component (e.g., Japan, Korea, China).
- Assembly Locations: Locations where the component is assembled (e.g., China, India, Vietnam).
- Terminal Node: Represents Apple HQ or a centralized manufacturing location.

Each edge includes:

- A cost (weight): includes base cost + tariff if applicable.
- Capacity constraints (upper and lower bounds): setted lowerbound to 1 to model one phone only.

Combined together, the vertices and edges are:

```
vertices = [
    'source',
    'Processor_Taiwan', 'Display_Korea', 'Battery_China',
    'Modem_China', 'Memory_US', 'Storage_Japan',
    'Camera_Supplier', 'Enclosure_Supplier', 'Misc_Suppliers',
    'Assembly',
    'Distribution_US',
    'terminal',
]
```

```

edges = [
    # From source to suppliers
    {'from': 'source', 'to': 'Processor_Taiwan', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Display_Korea', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Battery_China', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Modem_China', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Memory_US', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Storage_Japan', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Camera_Supplier', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Enclosure_Supplier', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Misc_Suppliers', 'weight': 0, 'lower': 1},

    # From suppliers to assembly (includes part cost as weight)
    {'from': 'Processor_Taiwan', 'to': 'Assembly', 'weight': 90.85, 'lower': 1},
    {'from': 'Display_Korea', 'to': 'Assembly', 'weight': 37.97, 'lower': 1},
    {'from': 'Battery_China', 'to': 'Assembly', 'weight': 5, 'lower': 1},
    {'from': 'Modem_China', 'to': 'Assembly', 'weight': 26.62, 'lower': 1},
    {'from': 'Memory_US', 'to': 'Assembly', 'weight': 21.80, 'lower': 1},
    {'from': 'Storage_Japan', 'to': 'Assembly', 'weight': 25.53, 'lower': 1},
    {'from': 'Camera_Supplier', 'to': 'Assembly', 'weight': 157.42, 'lower': 1},
    {'from': 'Enclosure_Supplier', 'to': 'Assembly', 'weight': 20.79, 'lower': 1},
    {'from': 'Misc_Suppliers', 'to': 'Assembly', 'weight': 200.06, 'lower': 1},

    # From assembly to U.S. distribution and customers
    {'from': 'Assembly', 'to': 'Distribution_US', 'weight': 0, 'upper': 100}, # shipping
    {'from': 'Distribution_US', 'to': 'terminal', 'weight': 0, 'upper': 100}, # local d
]

```

Running the program gives us:

optimal value: 549.73

The exact amount provided by WSJ proving that the system is functional.

4 Made in China

According to this method, the profit rate of an iPhone 16 pro based on its base price in the USA, which is \$999 on Apple's official website, will be:

$$\frac{999 - 549.73}{549.73} = 0.817 = 81.7\%$$

Now, assuming that this iPhone is made in China by certain companies like Foxconn, it will be charged a 34% tariff, resulting in its price being:

$$549.73 \times 1.34 = 736.63$$

With the price, the profit rate of the same iPhone 16 pro under tariff will be:

$$\frac{999 - 736.63}{736.63} = 0.356 = 35.6\%$$

To keep the same profit rate of 81.7%, the new price will have to be raised to:

$$736.63 + 736.63 \times 0.817 = \$1338$$

A same percentage of increase of tariff.

5 Made in USA

What if all parts are made in the manufacturing countries while the assembly is in USA? Then all parts will be charged with the tariff accounted for each of the country. For the minor components, we assume an average 20% tariff. In such a way, we modify the edges:

```
edges = [
    # From source to suppliers
    {'from': 'source', 'to': 'Processor_Taiwan', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Display_Korea', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Battery_China', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Modem_China', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Memory_US', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Storage_Japan', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Camera_Supplier', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Enclosure_Supplier', 'weight': 0, 'lower': 1},
    {'from': 'source', 'to': 'Misc_Suppliers', 'weight': 0, 'lower': 1},

    # From suppliers to assembly (includes part cost as weight)
    {'from': 'Processor_Taiwan', 'to': 'Assembly', 'weight': 119.92, 'lower': 1},
    {'from': 'Display_Korea', 'to': 'Assembly', 'weight': 37.97, 'lower': 1},
    {'from': 'Battery_China', 'to': 'Assembly', 'weight': 47.46, 'lower': 1},
    {'from': 'Modem_China', 'to': 'Assembly', 'weight': 35.67, 'lower': 1},
    {'from': 'Memory_US', 'to': 'Assembly', 'weight': 21.80, 'lower': 1},
    {'from': 'Storage_Japan', 'to': 'Assembly', 'weight': 20.59, 'lower': 1},
    {'from': 'Camera_Supplier', 'to': 'Assembly', 'weight': 126.95, 'lower': 1},
    {'from': 'Enclosure_Supplier', 'to': 'Assembly', 'weight': 27.86, 'lower': 1},
    {'from': 'Misc_Suppliers', 'to': 'Assembly', 'weight': 240.06, 'lower': 1},

    # From assembly to U.S. distribution and customers
    {'from': 'Assembly', 'to': 'Distribution_US', 'weight': 0, 'upper': 100}, # shipping
    {'from': 'Distribution_US', 'to': 'terminal', 'weight': 0, 'upper': 100}, # local d
]
```

And we get the number 678.28 as a result. This roughly a 27% increase from the Pre-tariff total. Certainly, this is a very rough and conservative estimate since

the cost of assembly in the USA will be significantly higher than that of China. Moreover, even though if the cost of assembly in the USA matches that of China in one day, this iPhone will have to increase its price to:

$$678.28 \times 1.817 = \$1232$$

In order to keep the same profit rate.

6 Analysis

Using the network-flow model for an analysis provides benefits for route optimization, however, as global trade is complicated and especially unpredictable under trade war and the cost in each part is unfathomable due to company secrets, it is hard to use this model to simulate a case for alternative options other than the suppliers provided. If there is clearer data on the shipment of parts and especially assembly cost difference, it is easier to model this problem with a clearer answer.

Also, when accounting for the Assembly part of the network flow, the problem gets linear with the tariff rate being the only influential component other than cost for suppliers and price prediction results will always be the same rate as the rate in which tariff changes. Truly, supply chain is much harder to model than this.

Moreover, there are more reasons than low cost that Apple choose the supply chain as it is today. Quality on assembly and parts is not considered in this research, which is an important factor for a top-quality product company. Also, the transportation and other miscellaneous cost is not calculated as edges in this model. Thus the actual profit margin of an iPhone 16 pro may be higher, and while tariff will affect these miscellaneous cost (especially transportation), the actual profit margin may be further reduced.

7 Conclusion

This project used network flow modeling in analyzing the global supply chain of the iPhone 16 Pro, particularly in the context of newly imposed tariffs. By modeling each major component individually and integrating them into a comprehensive final assembly and delivery network, we are able to evaluate how cost structures shift under different geopolitical and economic scenarios.

The simulation reveals that tariffs significantly erode profit margins unless adjustments—either in pricing or supply chain strategy—are made. For instance, a 34% tariff on China-assembled iPhones reduces the profit margin from approximately 81.7% to 35.6%, suggesting that Apple may need to raise retail prices to maintain its margins. On the other hand, shifting assembly to the U.S., while potentially avoiding some tariffs, introduces its own cost challenges and still leads to a noticeable increase in total cost.

While this model simplifies many real-world complexities—such as quality control, logistics, and hidden costs—it provides a useful quantitative framework for making strategic supply chain decisions. Future models could incorporate dynamic shipping costs, risk factors, or time-based constraints to further enrich the analysis. Overall, the network flow approach gives both a theoretical and practical lens for evaluating global manufacturing under evolving economic policies.

References

Code used for network-flow model: runNetworkFlow(vertices, edges)

```
def sbv(index, size):
    return np.array([1.0 if i == index else 0.0 for i in range(size)])

def objective(edges):
    return sum([e["weight"] * sbv(edges.index(e), len(edges)) for e in edges])

def getIncoming(vertex, edges):
    return [e for e in edges if e["to"] == vertex]

def getOutgoing(vertex, edges):
    return [e for e in edges if e["from"] == vertex]

def isSource(vertex, edges):
    return getIncoming(vertex, edges) == []

def isSink(vertex, edges):
    return getOutgoing(vertex, edges) == []

def interiorVertices(vertices, edges):
    return [v for v in vertices if not (isSource(v, edges) or isSink(v, edges))]

def conservationLaw(vertex, edges):
    ii = sum([sbv(edges.index(e), len(edges)) for e in getIncoming(vertex, edges)])
    oo = sum([sbv(edges.index(e), len(edges)) for e in getOutgoing(vertex, edges)])
    return ii - oo

def conservationMatrix(vertices, edges):
    return np.array([conservationLaw(v, edges) for v in interiorVertices(vertices, edges)])

def lowerBound(edge):
    return edge.get('lower', -math.inf)

def upperBound(edge):
    return edge.get('upper', math.inf)
```

```

def ineqConstraints(edges):
    m = np.array([*sbv(edges.index(e), len(edges)) for e in edges if upperBound(e) != math.inf,
                  *[-sbv(edges.index(e), len(edges)) for e in edges if lowerBound(e) != -math.inf])
    b = np.array([*upperBound(e) for e in edges if upperBound(e) != math.inf],
                  *[-lowerBound(e) for e in edges if lowerBound(e) != -math.inf])
    return m, b

def runNetworkFlow(vertices, edges, maximize=False):
    obj = objective(edges)
    Aeq = conservationMatrix(vertices, edges)
    Aub, bub = ineqConstraints(edges)
    beq = np.zeros(len(interiorVertices(vertices, edges)))
    sgn = -1 if maximize else 1
    result = linprog(sgn * obj, A_eq=Aeq, b_eq=beq, A_ub=Aub, b_ub=bub)
    if result.success:
        optimal_value = sgn * result.fun
        return [f"Optimal Cost: ${optimal_value:.2f}"] + [
            (f"{e['from']} -> {e['to']}", float(result.x[edges.index(e)])) for e in edges
        ]
    else:
        return ["Optimization failed:", result.message]

```

News Outlets:

<https://www.simplymac.com/iphone/how-much-does-it-cost-to-make-an-iphone>

<https://nypost.com/2025/04/06/us-news/heres-how-trumps-tariffs-could-send-iphone-prices-over-1000/>

<https://www.wsj.com/tech/personal-tech/iphone-apple-tariffs-china-bb20c7a3?st=3EAAFG&refl>